
Contrasting optimization methods and their underlying paradigms of genetic and swarm simulations using a Rastrigin minimization problem

Ina Aeon

Abstract

Using the results and processes of iterative optimization, contrast the performance of Genetic and Swarm simulations in solving the Rastrigin minimization problem, discussing the paradigms behind the methods that define their differences and abilities.

1 Genetic Algorithms

1.1 Definition

A Genetic Algorithm, in its simplest form, is the process of iteratively refining pseudo-random variations of data that represent an approach to the given solution, using functions that mimic natural processes; data is spliced and combined as an equivalent to genetic crossover, or randomly changed as an equivalent to genetic mutation.

Initial variations are provided to initiate the “population”, although these are not particularly suitable. These "genetic" variations undergo multiple stages in which they are evaluated and selected based on their fitness. Various selection methods can be implemented to further enhance the fitness of the population, Such as tournaments and roulette wheels, which add additional pressures that are dependent on the fitness of the variation, or by complete chance.

Following the selection process, individuals not chosen are removed from the gene pool. Those selected reproduce, creating additional variations known as “offspring”, which are differentiated from their parents through mutation and crossover methods. The success of this process depends on the parameters provided and methods implemented; as some stagnate or fail to produce results, further optimization of parameters yields greater results.

1.2 Optimization

1.2.1 Methodology

When optimizing the genetic algorithm, an iterative approach was taken to perform parameter sweeps. Using the built-in modules `itertools` and `os`, the provided GA Python script was adjusted to include functions that create permutations of a given range of possible values, generating parameter sets in a wide range, conducting a wide parameter sweep in a large batch, which outputs to directories that are generated and labeled with the parameters given.

These large batches of parameter sweeps simplified data collection, resulting in a large dataset to analyze and derive the best solution from each batch. It would take a very long time to perform each experiment and continuously tweak parameter sets using only a single thread, so I implemented a multiprocessing pool. It is also important to note that, since the Rastrigin function is a minimization function, the target is to achieve the lowest possible fitness value.

The initial parameter sweep had values influenced by the parameter set provided within the given materials; this helped establish a baseline configuration possible without implementing different methods or further tweaking parameters. Data collection and plotting proceeded, and using `matplotlib`, a 2D graph was plotted by iterating over each CSV file that was outputted from the initial experiment.

1.2.2 Iterative Analysis

Although 2D, Figure 1 plots the average fitness of each iteration, from each parameter set within the sweep. The lowest best fitness value for this sweep was greater than 1 and showed convergence within the population as the best fitness value moved in the correct direction. An ideal solution would see the curve flatten towards zero faster in an exploration period. This would then allow for more iterations to push the overall best fitness further through harsh competition, when only mutating and crossing over become insufficient to push the fitness further and would begin to stagnate.

However, without a 3rd dimension, what causes the behavior shown is difficult to distinguish. Therefore, 3D graphs in particular were chosen to attempt to reveal further details involving which parameters were influencing positive behavior by plotting the data alongside different parameter values on the Z axis.

Various 3D graphs were plotted to reveal and observe which parameters influenced the positive behavior. The initial parameter sweep had only the default 100 iterations and 100 particles, with 3D graphs plotted with an axis of additional values, such as tournament size (Figure 3). This allows for an easy comparison between the convergence curves and the best fitness produced. It is important to note that though `mutpb` is the probability of mutating an offspring, and `cxpb` is the probability of crossover when passed into DEAP, the parameter set notates the rate of mutation as `pmut-ind`.

Through this plotting, a "best overall" set of values could be distinguished, convergence curves can be observed and compared in these graphs and used to establish if the population is moving in the correct direction at a higher rate or stability (the line is less thick, lower, and the curve begins earlier). Overall, the best tournament size was 10 because it showed a better curve, which was carried onto the next parameter sweep.

The second parameter sweep implemented lower mutation rates that did better with the best solutions produced from the previous set. I also made the choice to use a population size of 50 and 150 iterations, so the algorithm has more iterations to mutate and cross over with. It yielded an overall best value of >0.83 , an improvement from the initial parameter sweep. (Figure 4)

The preceding parameter sweep continued using a population size of 50 and 150, but implemented higher crossover and mutation. This set of parameters resulted in the best average fitness going back-overs, at a best value of >14 . Attempting to tweak the parameters to push the algorithm further without changing the methods beyond the second parameter sweep seemed to lack any further improvement and did not push the value any lower, actually resulting in the average fitness increasing, so in pursuit of further optimization, I opted to start changing and adjusting methods and their sub-parameters instead of just simulation parameters. (Figure 5) (Figure 6)

In an attempt to push the evolution of the genetic algorithm further, different selection and mutation methods were introduced; these separate methods require extra parameters to run or will otherwise throw an exception, so adjustments to parameter set generation and algorithm run functions were made, which handle the different methods and assign their parameters correctly, allowing the writing of additional versions of the parameter dictionaries with different methods implemented, whilst maintaining the functionality of permuting possible values. (Figure 6)

Changes to crossover methods in the fifth parameter sweep introduced simulated binary bounded crossover, which uses the bounds specified so as not to make changes that are out of the bounds of the problem, as low and up are set to -5.12 and 5.12 to represent the bounds of the problem.

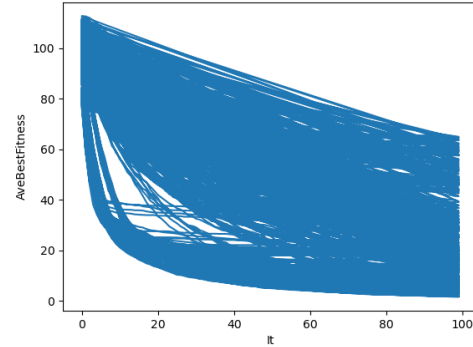


Figure 1: 2D Graph of the Initial GA Parameter Sweep

Crossing over or mutating too extremely to bounds outside of the problem results in best fitness values increasing rather than decreasing; Due to the solution being within a fixed range, this is important and resulted in the yield of an overall improved average best fitness rate across simulations, whilst also pushing the value even lower than the previous sweep; this operator allows for a wider range of diversity in crossover or the stable transfer of genetics, as the parameter eta specifies how much the offspring will resemble it's parents, this is important as lower values produce children with less of a resemblance to their parents, and higher values have more resemblance, this value must be balanced as to not randomize the result and stunt the development of better offspring, but also not prevent variation and exploration that could lead to better results. (Figure 7)

Polynomial Bounded Mutation was introduced in the sixth parameter sweep. Like simulated binary bounded crossover, it generates mutations within specified boundaries (low and up) and also uses the crowding distance parameter eta with higher values resembling a parent and lower values having less resemblance. This greatly improved overall average fitness and best fitness, clamping down the curve much harder by preventing runoff mutation outside of bounds that created stagnation or worse fitness values. There was an overall improvement, and the run produced a best value of 0.13. (Figure 8)

To conclude, the best overall final parameter set was created using the best parameter set from the sixth parameter sweep and consistently generates values smaller than 1, often reaching towards the results in the sixth sweep. The value produced is quite accurate; though there is likely further optimization possible. Due to the random nature of the genetic algorithm, the results cannot be perfectly reproduced. (Figure 2)

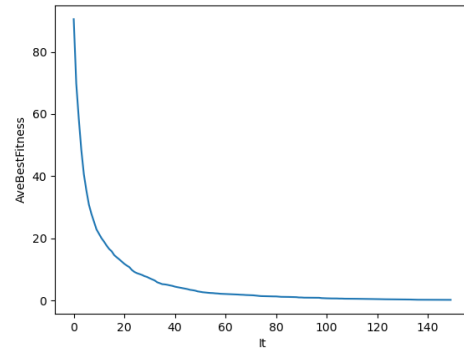


Figure 2: 2D Graph of the Final GA Parameter Set

2 Particle Swarms

2.1 Definition

Particle Swarms use the concept of particles in a space, where each particle has its own position and movement vector. These particles can adjust their position through this movement vector, gain or lose velocity through inertia, and can also retain their performance history for later comparison. Particles can communicate with each other and form social relationships and neighborhoods; these relationships can influence the next decision of a particle by sharing its best performance and vector.

There are topologies for establishing these relationships that can be selected which alter the way particles communicate and influence one another. As the particle is either pushed towards the position of its own overall best, or the best of the neighborhood, or the overall global best, it simulates social pressure that pushes a particle to converge with its neighbors. In some particle swarm simulations, an initial spike in exploration is implemented to allow the first iterations of particles to spread out and locate the direction of the best value, which can then influence the rest of the simulation and push the fitness value further.

2.2 Optimization

When optimizing the particle swarm, the additional code that was previously developed for the genetic algorithm was implemented to perform broad parameter sweeps. The best average fitness yielded from the initial run was around 11-12 (Figure 9)

The second parameter sweep achieved a best average fitness value of less than 10 through a slight adjustment to social and individual influence. (Figure 10)

The third parameter sweep changed the number of particles to 50 and iterations to 150, and the topology was changed to GlobalBestPSO, which influences each particle with the overall best throughout the population. This sweep yielded a best fitness >6 . (Figure 11)

The fourth parameter sweep adjusted inertia values and social pressure, yielding values <6 . There is good convergence using the inertia value 0.05; however, the less convergent curve (inertia=0.75) achieved greater average fitness. (Figure 12)

3 Conclusion

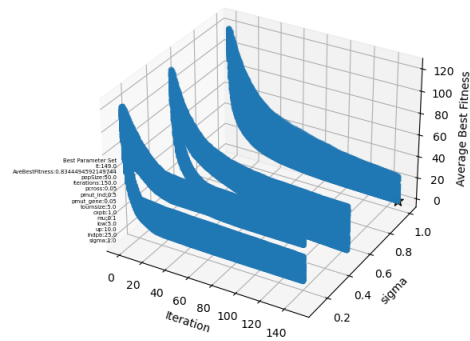
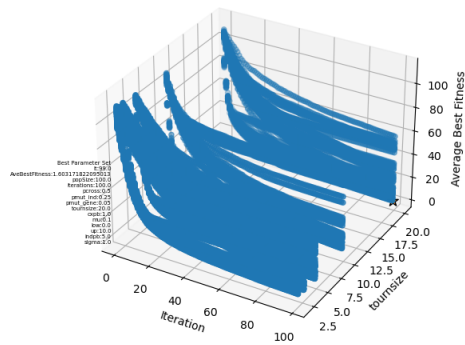
In contrast, the two algorithms share the same goal: to achieve the best possible fitness through constant adaptation. However, the two differ in the paradigms used to implement population control and pressures within the population to achieve a better outcome in the next iteration. Genetic algorithms are inspired by natural selection, whilst particle swarms use algorithms inspired by psychosocial behaviors in animal society.

The two differ in their method for retaining the previous best possible fitness with genetic algorithms using selection to push the overall best fitness further and crossover to retain that positive (or negative) information within the population. Particle swarms instead retain their previous best within each particle and communicate with one another to push their overall improvement (hence the name "swarm"), whilst also having additional inertia that pushes their exploration, which is similar to mutation seen implemented within a genetic algorithm; both introduce "randomness" in their own way.

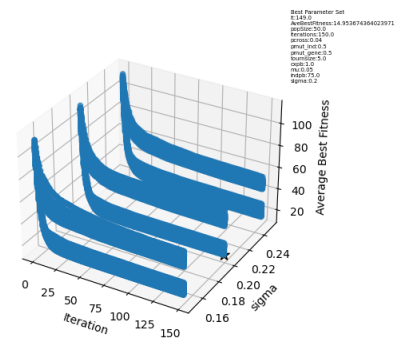
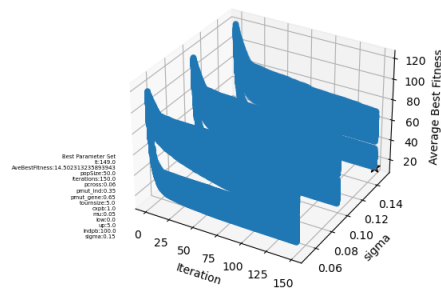
The two solutions differ in their potential ideal implementation, as genetic algorithms are better at handling chaotic data that by default is unbounded through constant mutation and crossover with evaluation to control the fitness for the solution; Ultimately, this can be unpredictable and cause stagnation or lead to little breakthrough though additional methods and optimization may improve performance, given just parameters without any specified additional methods, the genetic algorithm struggles in improving and finding the exact perfect solution.

In contrast to that, particle swarms are better suited to finding patterns within stable and non-erratic data, as inter-particle communication and information retention combined with the correct topology allow for a directed push towards the perfect solution, without the requirement of relying on mutation or crossover to occur "correctly" to attain an improved value. Due to this, optimizing a particle swarm to match a fixed set of data may be more efficient, where the best solution is in a static position, such as the reduction benchmark of the Rastrigin function.

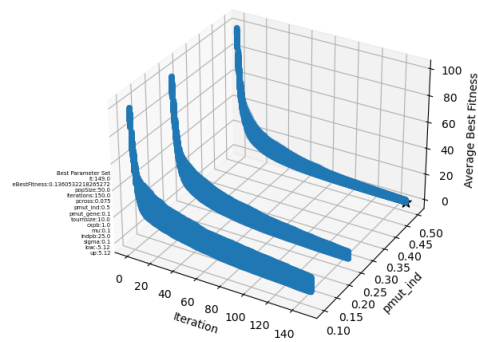
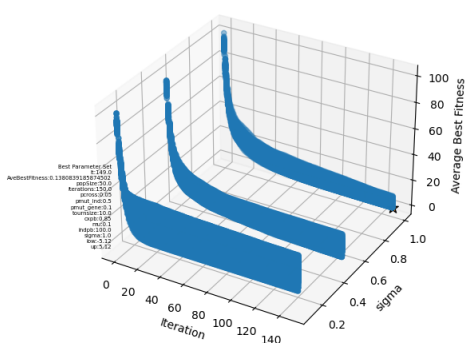
4 Illustrations and Figures



(3) 3D Plot of the initial GA sweep with tournament size plotted against average best fitness per iteration. (4) 3D Plot of the second GA sweep with sigma plotted against average best fitness per iteration.



(5) 3D Plot of the third GA sweep with sigma plotted against average best fitness per iteration. (6) 3D Plot of the fourth GA sweep with sigma plotted against average best fitness per iteration.



(7) 3D Plot of the fifth GA sweep with sigma plotted against average best fitness per iteration, it is important to note that the algorithm is using the crossover method `cxSimulatedBinaryBounded` and `mutPolynomialBounded`.

Plotted data of genetic algorithm parameter sweeps.

